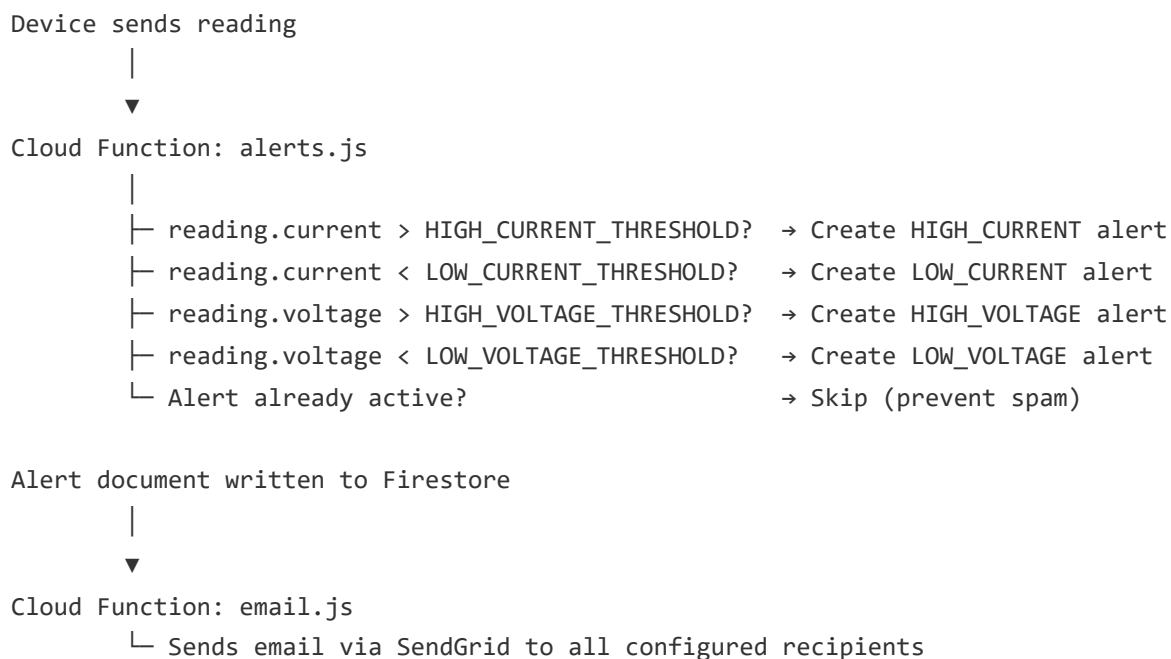


6 — Alerts & Scaling Guide

Audience: Developer / System Administrator / Operator **Covers:** Alert threshold configuration + adding 400+ stations efficiently

Part A — Alert System

How Alerts Work



Alert cooldown: A new alert of the same type is not raised again until the previous one is resolved. Resolution is automatic when the reading returns to the normal range.

Alert Threshold Configuration

Method 1 — Dashboard (Recommended for Operators)

1. Open the dashboard and navigate to a station.
2. Click the  **Settings** or **Edit** icon on the station.
3. Adjust the threshold fields:
 - **High Current (A)** — default: 18A
 - **Low Current (A)** — default: 2A
 - **High Voltage (V)** — default: 250V

- **Low Voltage (V)** — default: 200V

4. Save. The new thresholds take effect on the next reading (within 30 seconds).

Thresholds are stored in Firestore `stations/{stationId}` and read by the `alerts.js` Cloud Function on every incoming reading.

Method 2 — Firmware Defaults (Compile-time, for new boards)

Edit `firmware/src/config.h` before flashing:

```
#define DEFAULT_HIGH_CURRENT_THRESHOLD 18.0 // Amperes
#define DEFAULT_LOW_CURRENT_THRESHOLD 2.0 // Amperes
#define DEFAULT_HIGH_VOLTAGE_THRESHOLD 250.0 // Volts
#define DEFAULT_LOW_VOLTAGE_THRESHOLD 200.0 // Volts
```

These are only used if the Firestore document for the station has no thresholds set. The dashboard values always override firmware defaults.

Alert Debounce (Anti-Flap)

To prevent false alerts from single-sample spikes, the firmware includes a **3-sample debounce**:

- An edge condition must be present for **3 consecutive 5-second samples** before an alert payload is flagged in the telemetry upload.
- The server-side `alerts.js` function performs a **second check** against the actual reading value.

This means a transient spike shorter than ~15 seconds will not trigger an alert.

To change the debounce count, edit `firmware/src/config.h`:

```
#define ALERT_DEBOUNCE_COUNT 3 // Number of consecutive samples required
```

Alert Cooldown

After an alert fires, no second alert of the same type can be raised for **5 minutes**:

```
#define ALERT_COOLDOWN_MS 300000 // 5 minutes (in firmware)
```

On the server side, `alerts.js` also checks if an unresolved alert of the same type already exists in Firestore before creating a new one.

Alert Email Format

Alert emails include:

- Station name, ID, and GPS link
 - Alert type and severity
 - **Full reading table** at the time of the alert (Voltage, Current, Power, Energy, Frequency, Power Factor)
 - Timestamp (UTC)
 - Link to the dashboard station page
-

Part B — Scaling to 400+ Stations

Overview

The system is designed to scale linearly. No infrastructure changes are needed to go from 1 to 400+ stations:

- Firebase RTDB and Firestore scale automatically
 - Cloud Functions are serverless (auto-scale with load)
 - The web dashboard handles stations dynamically from the database
-

Bulk Provisioning Workflow

For large deployments, the most efficient workflow is:

[Laptop / Provisioning PC]

- |
- | 1. Admin creates 10 stations on dashboard → copies 10 tokens
- | 2. Technician updates `config.h` for each board (or batch script)
- | 3. PlatformIO flashes boards in sequence (USB-C cable)
- | 4. Boards deployed in cabinets → auto-appear on dashboard

Time per board: ~5 minutes (including firmware flash + verification) **Time for 100 boards:** ~8–10 hours (one technician)

Batch Firmware Patching Script

For bulk station provisioning, you can automate the `config.h` update and flash cycle using a PowerShell or Python script:

```
# Example: batch_provision.py
import subprocess
import json

stations = [
    {"id": "STATION_002", "token": "abc123..."},
    {"id": "STATION_003", "token": "def456..."},
    # ... more stations
]

template_config = open("firmware/src/config.h.template").read()

for station in stations:
    # 1. Write config.h
    config = template_config \
        .replace("{}STATION_ID{}", station["id"]) \
        .replace("{}TOKEN{}", station["token"])
    with open("firmware/src/config.h", "w") as f:
        f.write(config)

    # 2. Flash via PlatformIO (connect board first)
    input(f"Connect board for {station['id']} then press ENTER...")
    subprocess.run(["pio", "run", "--target", "upload"], cwd="firmware")
    print(f"✅ Flashed {station['id']}")
```

Firestore Index Requirements

At 400+ stations, ensure these Firestore composite indexes are deployed (already in `firestore.indexes.json`):

Collection	Fields Indexed	Query Use
history/{stationId}/readings	timestamp ASC	History page pagination
alerts	stationId ASC, createdAt DESC	Alert list per station
alerts	resolved EQ, createdAt DESC	Unresolved alert dashboard

Deploy indexes:

```
firebase deploy --only firestore:indexes
```

RTDB Data Retention

Live readings in the Realtime Database are **not archived indefinitely** — they are overwritten on each upload (single `/live/{stationId}` node per station).

Historical readings are stored in Firestore and **automatically purged after 90 days** by the `purge.js` Cloud Function (runs daily).

To change the retention period, update `purge.js` :

```
const RETENTION_DAYS = 90; // Change to desired number of days
```

Performance at Scale

Metric	1 Station	100 Stations	400 Stations
RTDB writes/min	2	200	800
Firestore writes/min	2	200	800
Cloud Function invocations/min	4	400	1,600
Dashboard map markers	1	100	400
Estimated monthly cost	<\$0.05	~\$3–5	~\$14–20

The dashboard uses Firestore **real-time listeners** for live data — at 400 stations, recommend switching to polling (30s interval) to reduce WebSocket connection overhead. Contact the developer team if scaling beyond 400 stations.

Adding a New Alert Type

To add a custom alert type (e.g., Low Power Factor):

1. **Backend** (`alerts.js`): Add a new condition block following the existing pattern
2. **Backend** (`email.js`): Add a new row/label for the alert type in the email template
3. **Frontend** (`types.ts`): Add the new `AlertType` to the union type
4. **Frontend** (`i18n.ts`): Add DE + EN translation strings for the new type
5. **Redeploy** functions: `firebase deploy --only functions`